Akademia Tarnowska
Wydział Nauk Technicznych
Katedra Informatyki
Dr inż. Marcin Skobel

Sieci neuronowe i uczenie głębokie: Projekt
Zajęcia projektowe 4: Trening modelu uczenia głębokiego

Streszczenie

Celem projektu jest zbudowanie i wytrenowanie modelu głębokiego uczenia dla wybranego zadania. Najważniejszą decyzją jest wybór strategii: uczenie modelu od podstaw, dostrojenie modelu wstępnie wytrenowanego (*fine-tuning*) lub zaprojektowanie modelu generatywnego. Student powinien uzasadnić wybór, zaimplementować model, przeprowadzić trening i ocenić uzyskane rezultaty.

1 Etap 1. Wybór problemu i strategii budowy modelu

Wybierz konkretne zadanie, określ rodzaj danych, oczekiwane wyjście modelu i sposób oceny jakości. Następnie zdecyduj, jak model zostanie zbudowany i wytrenowany.

Strategia	Kiedy ją wybrać?	Zadanie projektowe
Od podstaw (*from scratch*)	Gdy dostępny jest odpowiednio duży zbiór danych, domena jest specyficzna albo celem jest samodzielne zaprojektowanie architektury. Wymaga zwykle większej liczby danych i obliczeń.	Zaprojektuj warstwy, dobierz ich szerokość i głębokość oraz wytrenuj wszystkie parametry od losowej inicjalizacji.
Fine-tuning	Gdy istnieje model wstępnie wytrenowany na podobnych danych, a zbiór docelowy jest mały lub średni.	Wybierz model bazowy, wymień lub dostosuj warstwę wyjściową, a następnie zamroź część warstw albo dostrój cały model. Porównaj obie możliwości, jeśli pozwala na to budżet.
Model generatywny	Gdy celem jest tworzenie nowych próbek, rekonstrukcja, odszumianie lub synteza danych, a nie tylko przypisywanie etykiety.	Wybierz odpowiedni paradygmat: VAE (rekonstrukcja i przestrzeń ukryta), GAN (generator i dyskryminator), model dyfuzyjny (iteracyjne odszumianie) lub model autoregresyjny (generowanie sekwencji). Ustal, czy uczysz go od podstaw, czy adaptujesz model bazowy.

Ważne rozróżnienie: wybór modelu generatywnego dotyczy rodzaju zadania i celu uczenia, natomiast wybór *from scratch* lub *fine-tuning* określa sposób inicjalizacji i trenowania parametrów. Te decyzje należy uzasadnić osobno.

Wybierz rodzinę architektury adekwatną do danych:

- **MLP** – dane tabelaryczne i wektory cech;
- **CNN** – obrazy lub lokalne wzorce w sygnałach;
- **Transformer** – tekst, sekwencje i obrazy, gdy ważne są zależności między elementami;
- **1D-CNN / LSTM / Transformer** – szeregi czasowe i sygnały;
- **VAE / GAN / Diffusion** – generowanie lub rekonstrukcja danych.

Do wykonania: przygotuj krótkie uzasadnienie obejmujące: (1) problem i dane, (2) wybraną strategię oraz rozważoną alternatywę, (3) rodzinę i nazwę architektury, (4) ograniczenia danych i sprzętu.

2 Etap 2. Zbudowanie i sprawdzenie modelu

Zaimplementuj model w PyTorch lub TensorFlow/Keras. Nie wystarczy wywołanie gotowej biblioteki: należy wskazać, które elementy architektury zostały zaprojektowane, zmienione lub dostrojone.

W opisie architektury podaj:

- kształt wejścia i wyjścia, np. $[B, 3, 224, 224]$ dla batcha obrazów RGB;
- główne warstwy / moduły i ich rolę;
- liczbę parametrów wszystkich i trenowalnych;
- sposób inicjalizacji: losowa, wagi wstępnie wytrenowane, zamrożony ekstraktor cech lub częściowe dostrajanie;
- w przypadku modelu generatywnego: co stanowi wejście i wynik oraz co dokładnie model ma generować lub rekonstruować.

Liczbę parametrów policz w kodzie (osobno wszystkie i trenowalne). Przed treningiem wykonaj pojedynczy przebieg w przód i sprawdź kształt wyjścia oraz wartości NaN/Inf. Dla modelu wstępnie wytrenowanego podaj nazwę checkpointu, źródło i wprowadzone zmiany.

3 Etap 3. Konfiguracja i trening

Przygotuj rozłączne zbiory **Train/Validation/Test**. Zbiór walidacyjny służy do wyboru modelu i hiperparametrów; zbioru testowego użyj dopiero do końcowej oceny. Zapobiegaj przeciekowi informacji i stosuj augmentację wyłącznie na danych treningowych.

Ustal i uzasadnij najważniejsze ustawienia:

- **Funkcja straty:** np. `CrossEntropyLoss` dla klasyfikacji wieloklasowej, `BCEWithLogitsLoss` dla klasyfikacji binarnej/wieloetykietowej, `MSELoss`/`L1Loss` dla regresji; dla generowania dobierz stratę do wybranej metody, np. błąd rekonstrukcji i składnik KL w VAE, stratę przeciwną w GAN lub błąd predykcji szumu w modelu dyfuzyjnym.

- **Optymalizator:** np. AdamW jako punkt startowy lub SGD z momentum. Podaj learning rate, weight decay, wielkość batcha i maksymalną liczbę epok.
- **Metryki:** dobierz do celu. Dla klasyfikacji raportuj co najmniej accuracy i F1; dla regresji MAE/RMSE; dla rekonstrukcji np. MAE/PSNR/SSIM; dla generowania dobierz metryki odpowiednie dla danych i uzupełnij je oceną jakości próbek.

W treningu zastosuj i opisz przynajmniej jedną strategię kontroli uczenia: harmonogram learning rate, *warmup* (stopniowe zwiększanie learning rate na początku), *Early Stopping* lub zapis najlepszego checkpointu. Dla modeli Transformer i dużych modeli wstępnie wytrenowanych warmup może stabilizować początkowe kroki treningu.

Przykład konfiguracji w PyTorch: `AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)` oraz `CosineAnnealingLR(optimizer, T_max=max_epochs)`. Aktualizuj scheduler po każdej epoce. Jeśli stosujesz warmup, podaj liczbę kroków i sposób połączenia go z dalszym harmonogramem.

4 Etap 4. Monitorowanie i ocena rezultatów

Podczas treningu zapisuj stratę treningową i walidacyjną, wybrane metryki, learning rate oraz najlepszy checkpoint. Możesz użyć TensorBoard, Weights & Biases lub własnych wykresów.

Po zakończeniu uczenia:

1. przywróć najlepszy model wybrany na podstawie walidacji;
2. oceń go na zbiorze testowym, bez dalszego dostrajania hiperparametrów;
3. pokaż wykresy uczenia i przykłady wyników lub błędów;
4. krótko oceń, czy model spełnił cel i jakie są jego ograniczenia.

5 Wymagane rezultaty projektu

Oddaj: (1) notebook `.ipynb` lub skrypty `.py`, (2) plik `requirements.txt` lub opis środowiska, (3) opis danych i podziału Train/Validation/Test, (4) implementację modelu i uzasadnienie strategii *from scratch*/*fine-tuning*/model generatywny, (5) konfigurację treningu i najlepszy checkpoint, (6) krótki raport z liczbą parametrów, wykresami uczenia i wynikami testowymi.

6 Pytania do omówienia

1. Dlaczego wybrana strategia budowy modelu była lepsza od najważniejszej rozważanej alternatywy?
2. Czy wyniki wskazują na przeuczenie lub niedouczenie? Jakie zmiany w architekturze albo treningu warto sprawdzić?
3. Jak oceniasz jakość modelu dla wybranego zadania: czy sama metryka wystarcza, czy potrzebna jest również analiza przykładów?